



HELWAN NATIONAL UNIVERSITY

Faculty of Engineering

Department of Robotics & Mechatronics

IoT-Enabled Closed-Loop DC Motor Control System with Dynamic Braking

Subject: Electrical Drives

Course Code: POW2309

Student Name:

Student ID:

Yousef Ahmed Elbeltagy

922230111

Mohamad Sherif Shabrawy

922230084

Amr Sherif Maher

922230070

TO: Dr. Eng. Mostafa Rid

Table of Contents

Abstract	3
1. Introduction	3
2. System Architecture	3
2.1 Hardware Components	3
2.2 Software Architecture	4
2.3 Electrical Interface and Pin Configuration	7
3. Speed Measurement and Signal Processing	7
4. Advanced Feed-Forward Control with Adaptive Bias	8
4.1 Adaptive Bias Correction	8
5. PID Control Design	8
5.1 Noise Suppression and Stability	9
5.2 Integral Windup and Deadband	9
6. Safety and Protection Mechanisms	11
7. Dynamic Braking Circuit and Sensor-Based Emergency Protection	11
7.1 Relay Switching Arrangement	12
7.2 Braking Resistor Bank	12
7.3 MOSFET Relay Driver Circuit	12
7.4 System Wiring Diagram	13
7.5 VL6180X Sensor Connection and Operation	13
7.6 Emergency Braking Control Logic	14
7.7 Dynamic Braking Circuit Component Summary	14
7.8 Hardware Documentation — Photo Reference	15
8. IoT and Human-Machine Interface (HMI)	15
8.1 Blynk Virtual Pin Mapping	15
8.2 Input Arbitrator	15
9. Experimental Results and Discussion	15
10. Electrical Drives Concepts Demonstrated	16
11. Practical Industrial Applications	17
12. Limitations	17
13. Conclusion	17
14. Future Work	18
15. Appendix	19

Design and Implementation of an IoT-Enabled Closed-Loop Speed Control System for a DC Motor Using ESP32

Abstract

This project presents the design and implementation of an IoT-enabled closed-loop speed control system for a high-speed brushed DC motor using an ESP32 microcontroller. The system integrates encoder-based speed feedback, a feed-forward duty-cycle model, and a PID controller to achieve accurate speed regulation over a wide operating range (0–12,000 RPM). User interaction is supported through a 4×4 matrix keypad with dedicated function keys (A: speed ramp up, B: speed ramp down, C: direction toggle / configurable stop time, D: emergency brake), a potentiometer, a local LCD display, and a cloud-based Blynk interface, enabling both local and remote control.

Safety mechanisms including stall detection, overspeed protection, soft start/stop routines, direction change handling, relay-based dynamic braking, and VL6180X time-of-flight sensor-based automatic emergency obstacle detection are incorporated to ensure reliable and safe operation. The dynamic braking circuit employs two SPDT relays switched by IRLZ44N MOSFET driver circuits to connect the motor across a parallel braking resistor bank ($3 \times 1\Omega$, 100W each), converting the motor's kinetic energy into heat for rapid emergency stopping. Experimental results demonstrate stable speed tracking with near-zero steady-state error under constant load, and robust performance under dynamic setpoint changes.

1. Introduction

DC motors are widely used in industrial and embedded applications due to their simplicity, high torque-to-size ratio, and ease of speed control. However, achieving accurate speed regulation over a large operating range requires closed-loop control techniques that compensate for nonlinearities, load variations, and supply fluctuations.

This project focuses on the development of a smart motor control system that combines classical control theory with modern IoT capabilities. The system is designed to:

- Precisely regulate motor speed using encoder feedback
- Allow both local and remote control of speed and direction
- Provide real-time monitoring and safety protection
- Implement relay-based dynamic braking for emergency stopping
- Integrate automatic obstacle detection using a VL6180X time-of-flight sensor
- Demonstrate practical application of PID control and electrical drives concepts in embedded systems

2. System Architecture

2.1 Hardware Components

The system is composed of the following main hardware components:

- **775 Brushed DC Motor (12V)** — Capable of operating between 7,000 and 12,000 RPM with high torque output.
- **Cytron MD13S Motor Driver** — Provides PWM-based speed control and direction control with up to 13 A continuous current capability. Integrated over-current and thermal protection enhance system safety.
- **ESP32 DevKit Microcontroller** — Acts as the main controller, providing PWM generation, encoder interfacing, ADC readings, and Wi-Fi connectivity.
- **Rotary Encoder** — Provides quadrature pulse feedback for real-time speed measurement.
- **Buck Converter** — Steps down the 12 V supply to a regulated 5 V level for powering the ESP32.
- **User Interface Components:**
 - 4×4 Matrix Keypad for numeric speed entry, direction selection, speed ramping (A/B keys), direction toggle and configurable stop time (C key), and emergency braking (D key)
 - Potentiometer for analog speed control
 - I2C LCD for local feedback
- **Power Supply (12 V, 15 A)** — Supplies sufficient current for both motor operation and control electronics.
- **SONGLE SLC-12VDC-SL-C SPDT Relay (×2)** — 12V coil, 30A contact rating, SPDT configuration used to switch motor terminals between the driver and braking resistor bank.
- **IRLZ44N Logic-Level N-Channel MOSFET (×2)** — $V_{DS} = 60V$, $I_D = 50A$, $R_{DS(on)} = 0.028\Omega$. Used as relay coil drivers, directly controlled by ESP32 3.3V GPIO logic.
- **RX24-100W Aluminum Case Power Resistor 1Ω (×3)** — 100W continuous power rating each. Three in parallel form a 0.333Ω braking bank with 300W combined capacity.
- **1N4007 General Purpose Silicon Diode (×2)** — 1A, 1000V. Used as flyback protection across each relay coil.
- **TOF050C Module with VL6180X Distance Sensor** — Time-of-flight sensor with up to 200 mm detection range, connected via I2C for automatic emergency obstacle detection.

2.2 Software Architecture

The firmware is structured into modular subsystems:

- Input handling (keypad, potentiometer, Blynk)
- Speed measurement and filtering
- Feed-forward duty prediction
- PID control loop
- Safety monitoring (stall, overspeed, direction-change guard)
- Dynamic braking and sensor-based emergency protection
- Display and IoT synchronization

A periodic control loop is executed using a timer interrupt at 150 ms intervals.

The flowchart below illustrates the main system loop, showing the non-blocking architecture that handles Wi-Fi synchronization, potentiometer polling, and 4×4 keypad input processing. The keypad branch now includes dedicated paths for numeric speed entry (0–9), direction change (#), A/B speed ramping, C key configurable soft-stop, D key emergency dynamic braking, and * emergency reset.

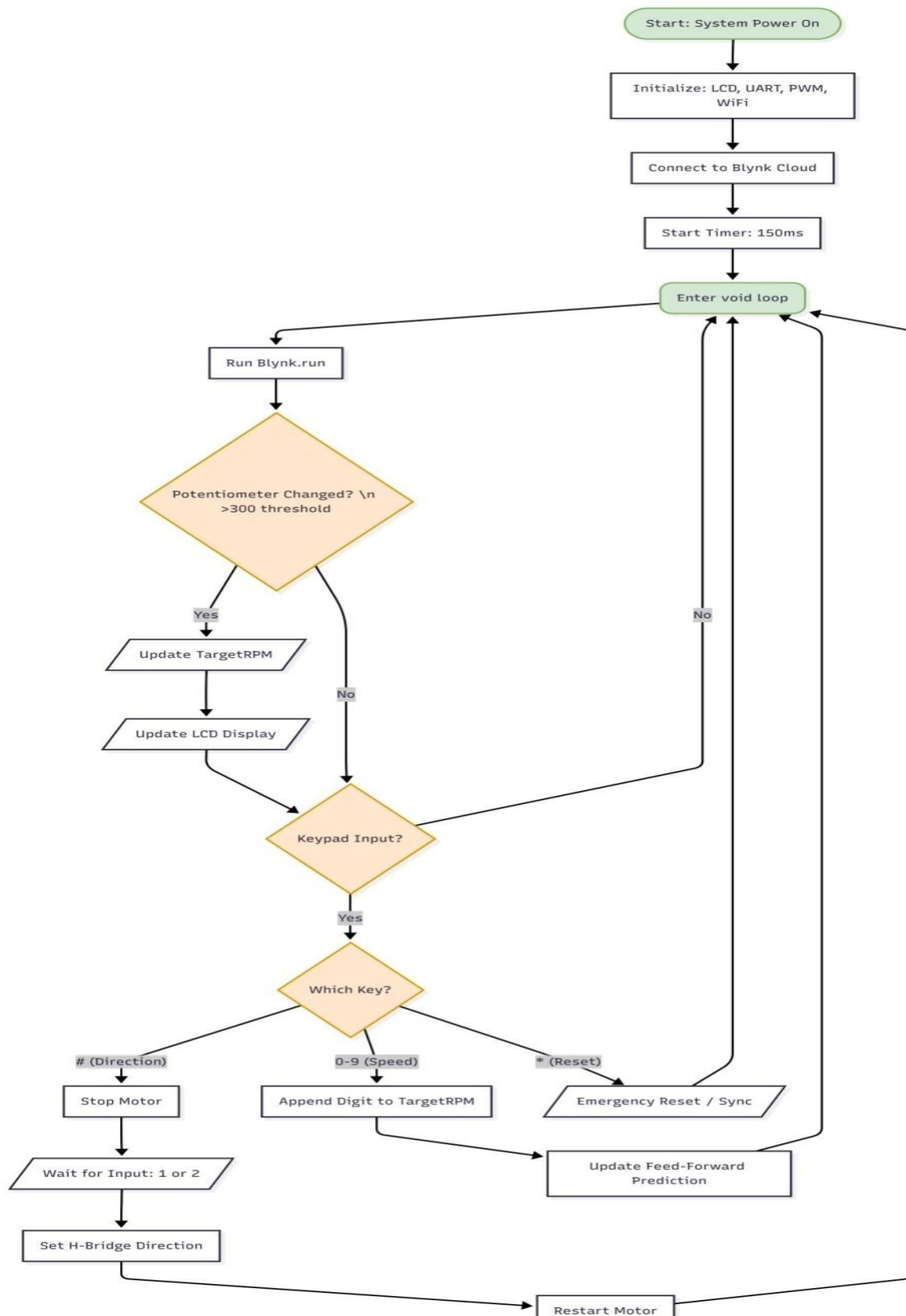


Figure 2.1: Main System Loop Flowchart. The non-blocking architecture handles Wi-Fi sync, potentiometer speed control (500 ADC count hysteresis), and 4×4 keypad inputs including speed ramp (A/B), configurable soft-stop (C), emergency dynamic braking (D key — activates relay braking circuit), and emergency reset (*). Direction changes (#) trigger a safe stop-then-reverse sequence.

2.3 Electrical Interface and Pin Configuration

To ensure hardware abstraction and modularity, the ESP32 GPIOs were configured with specific electrical characteristics. The system operates on a logic level of 3.3V, interfacing with the 12V motor subsystem via the motor driver.

- **PWM Generation:** The control signal is generated on GPIO 23 using the ESP32's LEDC peripheral. The PWM frequency is set to 20 kHz to minimize audible motor whine, with a resolution of 10 bits (0–1023 duty cycle steps), allowing for fine-grained voltage control.
- **Direction Control:** Motor direction signal on GPIO 25, connected to the MD13S DIR input. A HIGH output commands forward rotation; LOW commands reverse.
- **Quadrature Encoder:** The encoder channels A and B are connected to GPIO 32 and GPIO 33 respectively. These pins are configured as INPUT_PULLUP and utilize hardware interrupts to capture state changes on both rising and falling edges, maximizing measurement resolution.
- **User Interface:**
 - Keypad: A 4×4 matrix keypad is mapped to GPIOs 19, 18, 5, 17 (Rows) and 2, 16, 4, 15 (Columns).
 - Potentiometer: Analog speed reference is read via GPIO 36 (VP) using the ESP32's ADC.
 - I2C LCD: Driven via standard SDA/SCL lines using the LiquidCrystal_I2C library at address 0x27.
- **Relay Driver Outputs:** GPIO 26 (Relay A) and GPIO 27 (Relay B) drive the IRLZ44N MOSFET gates. Both pins start LOW at power-on and are driven HIGH together only during an emergency braking event.
- **VL6180X Sensor:** Connected to ESP32 GPIO 21 (SDA) and GPIO 22 (SCL) via the I2C bus.

3. Speed Measurement and Signal Processing

Motor speed is measured using a quadrature encoder with a physical resolution of 7 pulses per revolution (PPR). By counting both rising and falling edges on channels A and B, the effective resolution is quadrupled to **28 counts per revolution (CPR)**.

The control loop operates at a fixed sampling interval (T_s) of **150 ms**. Instantaneous speed is calculated based on the pulse count accumulated over this interval:

$$\text{RPM} = \frac{\text{Pulse Count}}{\text{CPR}} \times \frac{60}{\Delta t}$$

$$(Eq. 1) \quad \text{RPM} = (\text{Pulse Count} / \text{CPR}) \times (60 / \Delta t)$$

To mitigate quantization noise and measurement jitter, a moving average filter is applied. The firmware utilizes a circular buffer of size $N = 12$, calculating the smoothed RPM as:

$$\text{RPM}_{\text{smooth}} = \frac{1}{12} \sum_{i=0}^{11} \text{RPM}_{\text{raw}}[i]$$

$$(Eq. 2) \quad \text{RPM}_{\text{smooth}} = (1/12) \times \sum \text{RPM}_{\text{raw}}[i], \quad i = 0, 1, \dots, 11$$

This specific buffer size was selected to balance signal smoothness against phase lag, ensuring the controller remains responsive to rapid setpoint changes.

4. Advanced Feed-Forward Control with Adaptive Bias

Due to the nonlinear relationship between PWM duty cycle and motor speed, a feed-forward duty-cycle map was experimentally derived. The map associates discrete RPM values with corresponding PWM duty values.

To linearize the plant model, a lookup table (DutyMap) containing approximately 40 distinct RPM-to-Duty operating points (ranging from 0 to 12,284 RPM) was experimentally derived. The system employs **Linear Interpolation** to calculate the base duty cycle (D_{base}) for any requested target speed between calibrated operating points:

$$D_{\text{base}} = D_0 + \alpha(D_1 - D_0)$$

$$(Eq. 3) \quad D_{\text{base}} = D_0 + \alpha(D_1 - D_0)$$

where α is the interpolation ratio between adjacent speed points.

4.1 Adaptive Bias Correction

A novel feature of this implementation is the **Adaptive Feed-Forward Bias**. Since the static lookup table cannot account for battery voltage drop or varying mechanical loads over time, the system implements an online learning algorithm. When the steady-state error persists between 10 and 500 RPM, the system dynamically adjusts a bias term (dutyBias) for the specific segment of the lookup table:

$$\text{Bias}_{\text{new}} = \text{Bias}_{\text{old}} + \gamma \times (\text{DutyPerRPM}) \times \text{Error}$$

$$(Eq. 4) \quad \text{Bias}_{\text{new}} = \text{Bias}_{\text{old}} + \gamma \times (\text{DutyPerRPM}) \times \text{Error}$$

Where γ is a learning rate of 0.02. This allows the controller to “learn” the changing system characteristics in real-time, effectively shifting the feed-forward curve to match current operating conditions.

5. PID Control Design

To eliminate steady-state errors and improve transient response, a PID controller is implemented on top of the feed-forward term. The control law is given by:

$$u(t) = D_{\text{base}} + K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

$$(Eq. 5) \quad u(t) = D_base + K_p \cdot e(t) + K_i \cdot \int e(t) dt + K_d \cdot [de(t)/dt]$$

where the error signal is defined as:

$$\bullet \quad e(t) = \text{RPM}_{\text{target}} - \text{RPM}_{\text{measured}}$$

$$(Eq. 6) \quad e(t) = \text{RPM_target} - \text{RPM_measured}$$

The feedback mechanism utilizes a discrete PID algorithm with the following parameters tuned for the 775 DC motor:

- $K_p = 0.02$
- $K_i = 0.00015$
- $K_d = 0.0006$

$$(Eq. 7) \quad K_p = 0.02, K_i = 0.00015, K_d = 0.0006$$

5.1 Noise Suppression and Stability

To prevent high-frequency noise from amplifying through the derivative term, a **Digital Low-Pass Filter (Exponential Moving Average)** is applied to the derivative calculation with a smoothing factor of $\alpha = 0.62$:

$$D_{\text{filtered}} = 0.62 \cdot D_{\text{prev}} + (1 - 0.62) \cdot D_{\text{raw}}$$

$$(Eq. 8) \quad D_filtered = 0.62 \cdot D_prev + (1 - 0.62) \cdot D_raw$$

5.2 Integral Windup and Deadband

To ensure system stability, two additional mechanisms are implemented:

1. **Integral Windup Protection:** The integral term is strictly constrained between $-12,000$ and $+12,000$ to prevent integrator saturation during large error transients.

2. Deadband Zone: A deadband of ± 5 RPM is implemented. If the error falls within this range, it is treated as zero, preventing control output oscillation (chattering) around the setpoint.

The flowchart below shows the logic flow of the 150 ms Periodic Interrupt Service Routine (ISR). The ISR first prioritizes safety checks (stall detection and overspeed protection), then checks for VL6180X sensor-triggered emergency braking, before computing the feed-forward lookup, adaptive bias correction, and PID output.

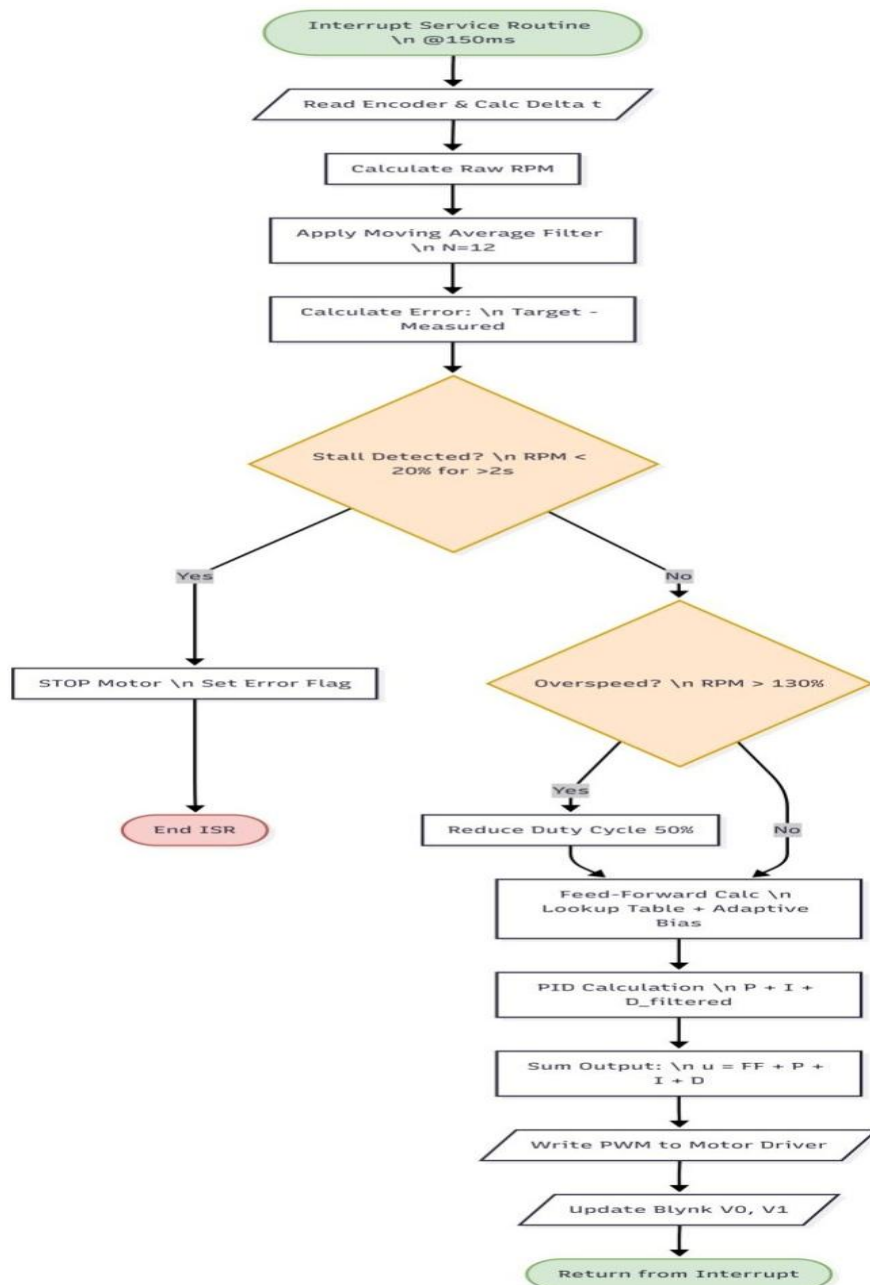


Figure 5.1: Periodic Interrupt Service Routine (ISR) executing every 150 ms. The routine prioritizes safety checks (stall detection: RPM < 20% of target for >2 s; overspeed: RPM > 130% of target for > 1 s) before executing the adaptive feed-forward lookup table, bias correction, and PID calculation ($P + I + D_{\text{filtered}}$). The VL6180X sensor emergency braking check is integrated at the safety stage, triggering relay-based dynamic braking when an obstacle is detected within the threshold distance.

6. Safety and Protection Mechanisms

Several protection strategies are implemented to ensure safe operation across all conditions:

- **Stall Detection:** The system monitors if the liveRPM falls below 20% of targetRPM. If this condition persists for 2000 ms (2 seconds) while the target is above 200 RPM, the motor is emergency stopped, and a “STALL ERROR” is triggered on the local display.
- **Overspeed Protection:** If liveRPM exceeds 130% of targetRPM for more than 1000 ms, the controller automatically halves the duty cycle to protect the mechanical assembly.
- **Soft Start / Soft Stop:** To reduce inrush current, changes in duty cycle are ramped. The ramp step size is dynamic, calculated as $\max(\Delta\text{Duty} / 30, 2)$, ensuring a smooth transition regardless of the speed jump magnitude.
- **Safe Direction Change:** Direction changes are only allowed after the motor has been brought to a complete stop, preventing excessive current spikes.
- **Relay-Based Dynamic Braking (D Key):** Pressing the D key activates two SPDT relays via MOSFET drivers, disconnecting the motor from the driver and connecting it across the braking resistor bank for rapid dynamic braking.
- **VL6180X Sensor-Based Automatic Emergency Braking:** The time-of-flight sensor continuously monitors for obstacles near the cutting tool at a detection threshold of 80 mm. When an object is detected within this distance for two consecutive readings, the system automatically triggers the same dynamic braking circuit. Sensor detection and braking activation occur within approximately 80–120 ms.
- **Configurable Soft-Stop Time (C Key – Motor Stationary):** Allows the operator to configure a soft-stop deceleration time between 1 and 5 seconds.
- **Live Direction Toggle (C Key – Motor Running):** Toggles between forward and reverse with a safe stop-and-restart sequence.
- **Hold-to-Ramp Speed Adjustment (A/B Keys):** Holding A ramps speed up in 10 RPM increments; holding B ramps it down.

7. Dynamic Braking Circuit and Sensor-Based Emergency Protection

The system was upgraded with a relay-based dynamic braking circuit to provide fast stopping of the DC motor during emergency conditions. Two 12V SPDT relays are used together to form a DPDT switching arrangement, with each relay controlling one motor terminal.

7.1 Relay Switching Arrangement

In normal operation, the relay common (COM) contacts are connected to the motor terminals, while the normally closed (NC) contacts link the motor to the motor driver. When braking is required, both relays energize simultaneously, moving the motor terminals from NC to normally open (NO) contacts and connecting the motor across the braking resistor bank. The contact wiring is:

- Relay A COM → Motor terminal 1
- Relay B COM → Motor terminal 2
- Relay A NC → Motor driver output terminal 1 (normal run path)
- Relay B NC → Motor driver output terminal 2 (normal run path)
- Relay A NO → One end of the braking resistor bank (braking path)
- Relay B NO → Other end of the braking resistor bank (braking path)

When the relays are de-energized the motor connects normally to the driver. When the relays are energized the motor is isolated from the driver and connected across the resistors.

7.2 Braking Resistor Bank

Three RX24-100W aluminum power resistors (1Ω each) are connected in parallel. The equivalent resistance and total power dissipation capacity are:

$$R_{eq} = 1 / (1/R_1 + 1/R_2 + 1/R_3) = 1 / (1/1 + 1/1 + 1/1) = 1/3 \approx 0.333 \Omega$$

(Eq. 9) Equivalent resistance of three 1Ω resistors in parallel

$$P_{total} = P_1 + P_2 + P_3 = 100 + 100 + 100 = 300 W$$

(Eq. 10) Total rated power dissipation of the braking resistor bank

During braking, the rotating DC motor acts as a generator and its kinetic energy is dissipated as heat in the resistor bank. The low equivalent resistance (0.333Ω) maximizes braking current and consequently the braking torque, producing a significantly faster stop than PWM soft-stop alone.

7.3 MOSFET Relay Driver Circuit

Each relay coil is driven by an IRLZ44N MOSFET, a logic-level device that can be fully turned on by the ESP32's 3.3V GPIO output. The wiring for each driver stage is:

- ESP32 relay control GPIO → 100–220Ω gate resistor → IRLZ44N gate
- 10kΩ pull-down resistor from MOSFET gate to GND (prevents false triggering)
- IRLZ44N source → common GND (ESP32 GND and 12V supply negative)

- IRLZ44N drain → relay coil pin 1
- Relay coil pin 2 → +12V supply
- 1N4007 flyback diode across relay coil: cathode → +12V, anode → MOSFET drain

This driver stage is repeated identically for Relay A and Relay B. Both relay control pins are activated together by the ESP32 during any emergency braking event.

7.4 System Wiring Diagram

The figure below shows the complete system wiring diagram, illustrating the relay switching paths, MOSFET driver circuits, braking resistor bank, sensor connections, and the normal-run vs. braking power paths.

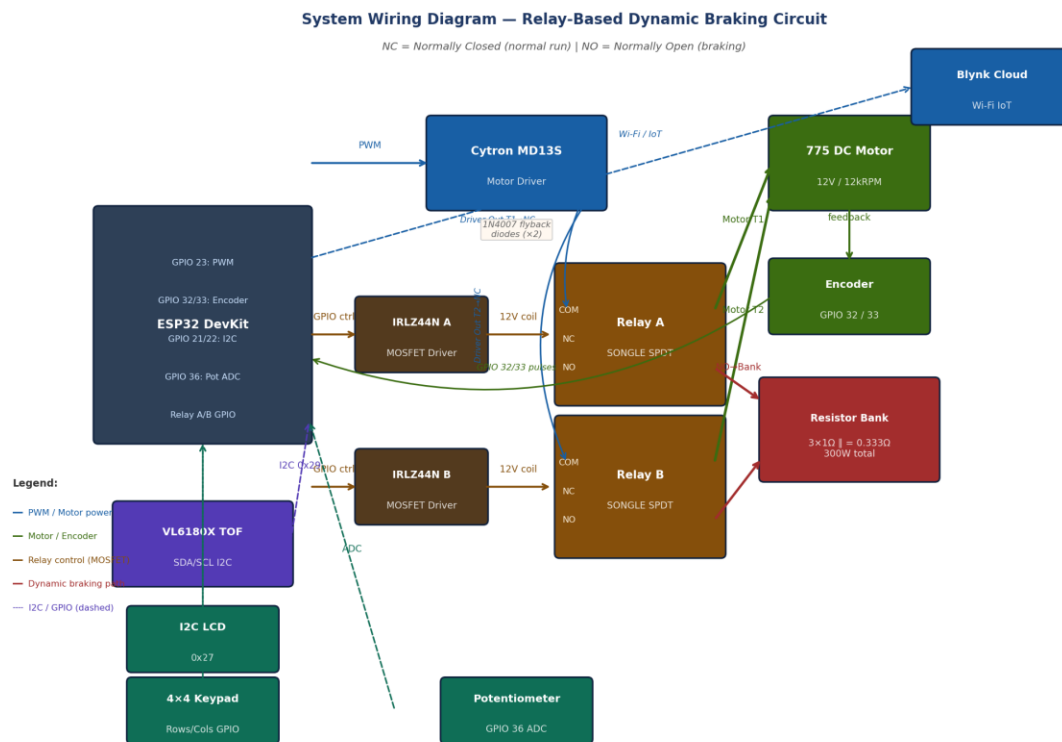


Figure 7.1: Complete System Wiring Diagram. Blue = PWM/driver paths, Green = motor/encoder, Amber = relay coil control, Red = dynamic braking path, Dashed = I2C/GPIO signals.

7.5 VL6180X Sensor Connection and Operation

A VL6180X time-of-flight distance sensor is added as a safety sensor to detect a hand or object near the rotating cutting disc. The sensor is connected to the ESP32 using I2C:

- VL6180X VCC → 3.3V (or 5V depending on module version)
- VL6180X GND → ESP32 GND

- VL6180X SDA → ESP32 GPIO 21
- VL6180X SCL → ESP32 GPIO 22

The sensor is mounted near the cutting tool tip at a fixed height above the workpiece surface. The firmware uses a detection threshold of 80 mm (defined as `OBSTACLE_DISTANCE_MM` in the code): if the measured distance drops to 80 mm or below for two consecutive readings, the emergency braking routine is triggered. During normal operation the workpiece sits below this threshold. If a finger or hand enters the detection zone, the ESP32 immediately zeros the motor PWM and energizes both relay coils, connecting the motor to the braking resistor bank. Sensor detection and braking activation occur within approximately 80–120 ms, providing a critical safety margin for operator protection.

7.6 Emergency Braking Control Logic

The system has two braking trigger methods:

1. Manual emergency braking: Pressing the D key on the 4×4 keypad activates the relay-based dynamic braking circuit.
2. Automatic sensor braking: If the VL6180X detects an object within the threshold distance, the system automatically activates the same circuit.

After either braking event, the operator must press the * (star) key to reset the emergency state and de-energize the relays, returning the system to normal operating mode. This deliberate reset requirement prevents accidental restart after an emergency stop.

7.7 Dynamic Braking Circuit Component Summary

#	Component	Qty	Key Specification
1	SONGLE SLC-12VDC-SL-C Relay	2	12V coil, SPDT, 30A contacts
2	IRLZ44N N-Channel MOSFET	2	60V, 50A, $R_{DS(on)} = 0.028\Omega$
3	RX24-100W Power Resistor	3	1 Ω , 100W, aluminum case
4	1N4007 Diode	2	1A, 1000V, flyback protection
5	TOF050C / VL6180X Sensor	1	I2C, 0–200 mm range
6	Gate Resistor	2	100–220 Ω , MOSFET gate current limiting
7	Pull-Down Resistor	2	10k Ω , MOSFET gate discharge

Figure 7.2: Dynamic Braking Circuit Component Summary Table

8. IoT and Human-Machine Interface (HMI)

The system operates a dual-interface architecture allowing synchronized control via physical hardware and the Blynk IoT Cloud.

8.1 Blynk Virtual Pin Mapping

- **V0:** Live RPM Telemetry (Output to Graph)
- **V1:** Target RPM Setpoint (Input/Output sync)
- **V2:** Direction Control (0 = Forward, 1 = Reverse)
- **V3:** Master Start/Stop Switch
- **V4:** Remote LCD Widget (Mirrors physical LCD)

8.2 Input Arbitrator

The firmware acts as an arbitrator between inputs. The physical keypad allows the user to interrupt operation to change direction. The potentiometer input utilizes a 15-sample moving average to filter electrical noise; control authority is granted to the potentiometer only when a change in value exceeds a hysteresis threshold of **500 raw ADC counts**, preventing drift from overriding IoT commands.

9. Experimental Results and Discussion

Experimental testing was performed to validate system performance across multiple operating scenarios. The following table summarizes key test conditions and observations:

Test Condition	Observation
Normal speed tracking	Stable RPM tracking with very low steady-state error across the full 0–12,000 RPM range
Soft stop (1–5 s)	Motor decelerates smoothly over the operator-selected time; no sudden torque spikes observed
Dynamic braking (D key)	Motor stops significantly faster than soft stop; relay switching and resistor heat dissipation confirmed
Sensor detection	Emergency braking activates automatically when obstacle detected; response time ~80–120 ms
Direction reversal	Motor safely stops before reversing direction; no excessive current spikes observed
A/B hold ramp	Smooth incremental speed control achieved with 10 RPM steps per press
Stall detection	Motor correctly identified stall condition and shut down within 2-second timeout

Overspeed protection	Duty cycle automatically halved when RPM exceeded 130% of target for >1 s
-----------------------------	---

Figure 9.1: Experimental Results Summary

Braking Method Comparison

Soft Stop: The motor decelerates gradually by ramping the PWM duty cycle over the configured 1–5 seconds. This produces minimal mechanical stress but results in a longer stopping distance, which may be unacceptable in safety-critical situations.

Dynamic Braking (D Key): When the operator presses D, the relays switch the motor from the driver to the braking resistor bank. The motor acts as a generator and its kinetic energy is rapidly dissipated as heat. This achieves a significantly faster stop than the soft stop method.

Automatic Emergency Braking (Sensor): The VL6180X sensor triggers the same relay-based dynamic braking circuit automatically. The total response time from sensor detection to relay activation is approximately 80–120 ms under typical operating conditions, providing a meaningful safety margin for operator protection.

The combination of feed-forward control and PID feedback significantly improved response time compared to a pure PID approach. Near-zero steady-state error was consistently observed under constant load conditions, and the system demonstrated robust performance under dynamic setpoint changes.

10. Electrical Drives Concepts Demonstrated

This project demonstrates several fundamental electrical drives concepts in a practical embedded system context:

- PWM motor speed control using variable duty-cycle voltage regulation
- Closed-loop feedback control with encoder-based speed measurement
- PID control for steady-state error reduction and transient response improvement
- Dynamic braking through resistive kinetic energy dissipation
- Back-EMF energy recovery and dissipation using a parallel resistor bank during generator-mode operation
- Braking torque generation proportional to motor speed and inversely proportional to braking resistance
- Safe direction reversal with mandatory stop-before-reverse sequencing
- Embedded motor protection including stall detection and overspeed limiting
- Human safety integration using time-of-flight proximity sensing for automatic emergency braking

11. Practical Industrial Applications

The system architecture and safety features demonstrated in this project have direct applicability to several industrial and educational domains:

- CNC cutting systems requiring precise speed control and rapid emergency stop capability
- Conveyor belt systems where speed regulation and stall detection are critical
- Automated machine tools with operator proximity safety requirements
- Smart workshop safety systems integrating IoT monitoring with local hardware protection
- Educational motor drive trainers for teaching closed-loop control, PID tuning, and dynamic braking principles

12. Limitations

While the system performs well as a prototype, several limitations should be acknowledged for future improvement:

- Relay switching delay: Mechanical relays introduce a finite switching delay (typically 5–15 ms) compared to solid-state solutions such as MOSFET H-bridges, which switch in microseconds.
- Heat generation in braking resistors: During repeated or prolonged braking events, the aluminum-cased resistors accumulate significant heat. Active cooling may be required for continuous industrial use.
- Sensor calibration dependency: The VL6180X detection threshold must be carefully calibrated for each workpiece height. Changes in workpiece geometry require recalibration.
- Prototype-level safety system: While functional, this is a demonstration-grade implementation. Industrial deployment would require redundant sensors, fail-safe relay configurations, and compliance with relevant machine safety standards.
- Mechanical relay wear: Electromechanical relays have a finite contact life (typically 100,000 operations). High-frequency braking applications would benefit from solid-state relay alternatives.

13. Conclusion

This project successfully demonstrates a robust, IoT-enabled DC motor speed control system that integrates classical control theory with modern embedded and cloud technologies. The use of feed-forward modeling, adaptive bias correction, and PID control results in high accuracy, stability, and near-zero steady-state error under normal operating conditions.

The addition of a relay-based dynamic braking circuit, employing two SPDT relays switched by IRLZ44N MOSFET drivers and a parallel braking resistor bank ($3 \times 1\Omega$, 300W), enables rapid emergency stopping by converting the motor's kinetic energy into heat. The integration of a VL6180X time-of-flight sensor provides automatic obstacle detection, triggering the dynamic braking circuit without operator intervention when a hand or foreign object is detected near the cutting tool. Sensor detection and braking activation occur within approximately 80–120 ms.

The upgrade to a 4×4 matrix keypad provides dedicated function keys for speed ramping, direction toggling, configurable soft-stop timing, and one-press emergency braking. The system architecture is scalable and can be extended to higher-power motors, additional sensors, or edge-based AI monitoring in future work.

14. Future Work

To elevate this system from a prototype to an industrial-grade solution, the following technical extensions are proposed:

- **Model-Based Control & System Identification:** Future iterations will implement mathematical modeling to derive the motor's transfer function parameters (inertia, friction, back-EMF). This enables advanced control strategies like Linear Quadratic Regulation (LQR) or Model Predictive Control (MPC) to optimize performance beyond standard PID limitations.
- **Edge-AI Processing on Single Board Computers:** Migrating high-level processing to a Raspberry Pi would allow for neural network deployment, enabling features such as real-time PID auto-tuning and intelligent load classification, while the ESP32 remains dedicated to real-time driver actuation.
- **Industrial Communication Protocols:** To ensure reliability in electrically noisy environments, the current Wi-Fi interface should be supplemented with RS-485 (Modbus) or CAN Bus. These differential signaling protocols provide superior immunity to Electromagnetic Interference (EMI) and allow direct integration with industrial PLCs.
- **Predictive Maintenance via Signal Analysis:** By integrating current sensors and accelerometers, the system could perform Fast Fourier Transform (FFT) analysis on vibration and current signatures, allowing early detection of mechanical anomalies such as bearing wear or misalignment before catastrophic failure occurs.
- **Multi-Sensor Fusion for Enhanced Safety:** Future versions could incorporate LiDAR, thermal imaging, or multiple TOF sensors to provide comprehensive operator safety detection beyond the current single-point VL6180X range.
- **Regenerative Braking:** Implementing regenerative braking to recover kinetic energy into a storage element (supercapacitor or battery) instead of dissipating it as heat would improve energy efficiency, particularly in applications with frequent start-stop cycles.

15. Appendix

15.1 Firmware Source Code

The complete firmware source code for the ESP32 controller is hosted in a digital repository. This includes the PID algorithm, the feed-forward linearization map, the Blynk IoT integration logic, the dynamic braking relay control, and the VL6180X sensor-based emergency braking system.

Access Instructions: Scan the QR code below using a smartphone camera to view the full C++ source file, or visit the direct link provided.



Figure 15.1: QR Code linking to the Closed_Loop_Motor_Control.cpp source file.

Direct Repository Link:

https://drive.google.com/drive/folders/18FHJYEo4qCkck1h6DbH2Hz4GTF137ZH?usp=drive_link